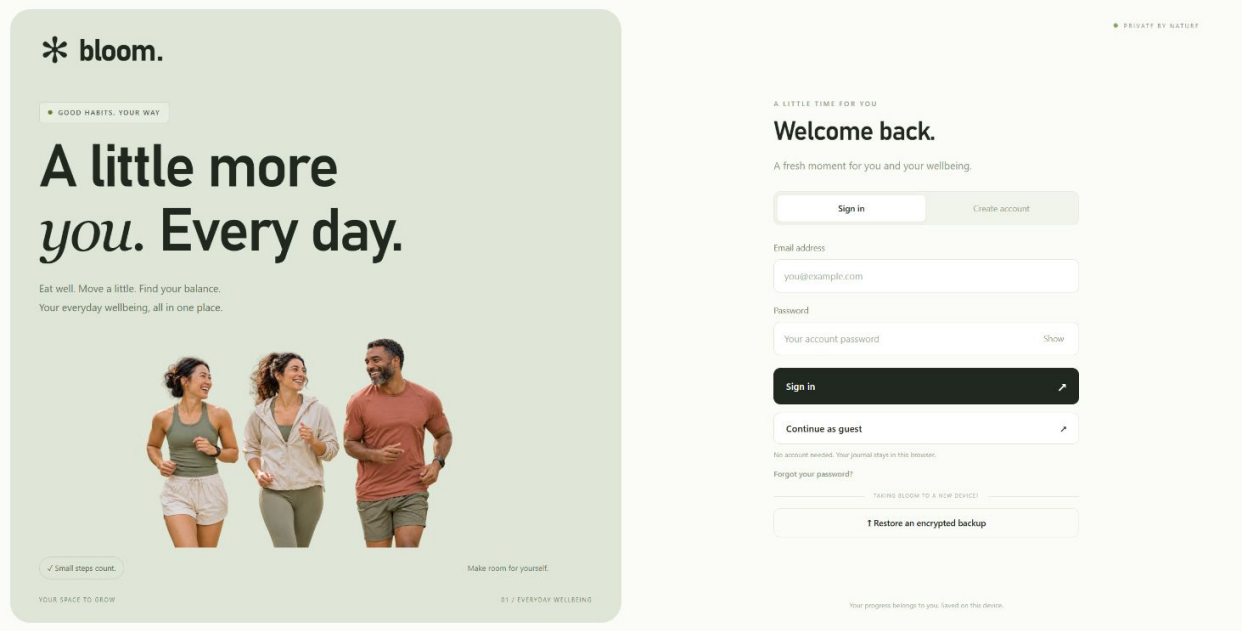
bloom.

A LITTLE MORE YOU. EVERY DAY.

The complete application guide

A local health journal, an optional photo assistant, and a reproducible machine learning project.



Use it. Understand it. Share it.

Product features | Setup and sharing | Privacy and backups
AI and ML workflows | Architecture | Testing and next steps

Implementation snapshot: 08 November 2026 | Share package 6

01 / START HERE

What Bloom is

A personal journal for food, hydration, movement, goals and progress, with optional AI assistance.

Bloom runs in your browser through a small server on your own computer. You can use it as a guest or create a password-protected local account. Its main job is to help you record what you eat and do, then show totals from those entries.

What works now

Guest and local account access, meal and ingredient entry, calorie/macronutrient totals, optional micronutrients, water, activity, weight entries, goals, progress views and portable backups. The app is currently a local development application.

Capability	Current state
Online photo analysis	Implemented OpenAI connection; requires your own API key and internet. Live recognition accuracy has not been established.
Trainable local food model	Training, evaluation and serving code implemented. No real-food model is trained or bundled yet.
ML verification	Synthetic training/inference and browser correction flow passed. Synthetic models are blocked from ordinary food use.
Sharing	Send the ZIP; your friend runs a separate local copy. No public hosted URL, installer or cloud syncing is included.

How to use this guide

Pages 3-7 explain setup, sharing and everyday use. Pages 8-9 explain data handling and implementation. Pages 10-12 cover the ML project. Pages 13-16 record verification, career use, troubleshooting and references.

The visual design uses restrained sage, charcoal and warm neutral surfaces. The login image has a transparent background, and the login page fits without scrolling at the tested screen sizes. The 21st.dev design patterns were adapted to native HTML, CSS and JavaScript.

02 / SHARE & INSTALL

Share it with a friend

Send the application package, not the localhost address on your computer.

The file to send

bloom-health-v6.zip. It contains the application source, Windows launcher, user documentation, this PDF and the optional ML pipeline. Personal journals, account backups, API keys, datasets and trained weights are excluded.

What you do

- Send the ZIP by email attachment, a messaging app, USB drive, or a private Drive/OneDrive/Dropbox link. Share the PDF too if your friend wants to read before installing.
- You do not need to share your own account, password or journal. Sharing the app gives your friend a separate copy with their own browser storage.

What your friend does on Windows

1. Install Python 3.10 or newer if it is not already installed. Make sure the python command is available.

2. Save the ZIP, right-click it and choose Extract All. Open the extracted bloom-health folder; do not run directly inside the ZIP.

3. Double-click Open Bloom.cmd. Keep its server window open while using the app.

4. Open http://localhost:4317 in a recent browser. Choose Continue as guest or create a local account.

5. Use the same browser profile and address when returning. Download a backup regularly.

Important details

localhost always means the computer opening the address. Your friend cannot reach your copy by opening your localhost link on their own device. The server deliberately listens on loopback only. Remote multi-user hosting would need a separate deployment and security design.

No Node installation or frontend build is required for ordinary use. Online AI and ML setup are optional. On macOS/Linux, the Windows launcher does not apply; Python can run python3 server.py from the extracted folder, but those platforms have not been verified here.

03 / ACCESS & BACKUPS

Your account. Your data.

Guest access is available below Sign in. Accounts are optional and local.

Choice	Behavior
Continue as guest	No credentials. Automatically saves a shared guest journal in this browser. Other people using the same browser profile can open it.
Create a local account	Uses a name, email label and password of at least 12 characters. Email is a local identifier; no verification email is sent.
Sign in elsewhere	Copy and restore an encrypted backup first. There is no central account service or automatic device synchronization.

Back up a guest journal

Open Backup or Backup & transfer and choose Download guest backup. It is an ordinary unencrypted JSON file. On another device, open an empty guest journal and import that file. Preserve any existing journal before attempting a replacement or migration.

Back up a password-protected account

Choose Download encrypted backup and save the .bloom file outside the browser. On the other device, choose Restore an encrypted backup on the sign-in page, select the file, enter the original password, review it, and choose Restore journal and sign in.

Backups contain profile and goals, meals and ingredients, portions and nutrient estimates, water, activity and weights. Logged model provenance is preserved when present. Photos and API keys are not part of journal backups. Email and backup metadata remain visible; journal contents are encrypted.

Keep the password and the latest backup

There is no password reset or recovery service. Clearing browser data removes local journals and accounts. Backups are snapshots; there is no automatic merging. Restoring over an existing copy requires confirmation and can replace newer data.

Creating an account does not silently move guest data. The original journal can be reviewed and moved into an empty account explicitly; it is encrypted and saved before the old unprotected copy is removed. Guest and account journals remain separate until that action.

04 / EVERYDAY USE

A journal that adds up

Seven navigation areas organize the daily workflow. Change the journal date to review earlier entries.

Page	What you can do
Overview	See calories, hydration, movement minutes and protein against your goals. Use quick actions for meals, photo review and activity.
Food journal	View meals grouped by Breakfast, Lunch, Dinner and Snack, plus macro balance and known micronutrients.
Photo meal	Choose/upload a photo or use the camera, request analysis, review ingredients, adjust portions and log consumption.
Activity	Record movement duration and notes; see the separate consumed-meal feed.
Progress	Review a seven-day calorie chart, logged-day statistics, movement and weight entries.
My goals	Edit profile name and goals for calories, protein, carbs, fat, water and activity minutes. Export or clear journal data deliberately.
Backup & transfer	Download or restore backups, review guest/account options and explicitly migrate supported older journals.

Small actions, clear meaning

Water controls add or remove 250 ml. Activity records minutes, not measured energy expenditure. Weight is an optional manual entry. Removing a meal updates its totals. The app shows empty states when nothing has been logged, and any example journal is labeled as sample data.

Goal percentages show progress toward the numbers you set. Remaining calories are the calorie goal minus logged food intake, with the displayed remainder bounded at zero. Activity minutes are not automatically converted into calories or subtracted from intake.

Mobile layouts use bottom navigation. Dialogs allow entry and review without losing the main page. Reduced-motion preferences and visible keyboard focus are supported. The app does not include wearable syncing, step counting or automatic body measurements.

05 / FOOD & NUTRIENTS

Know what is being counted

Nutrition totals come from reviewed ingredient values and portions, not a general assessment of health.

Two ways to enter food

- Quick Add food: enter a dish, meal type, calories, protein, carbs and fat from your label or recipe.
- Ingredient review: use Add missing food or ingredient, select a starter food or enter your own values per 100 grams, and enter the portion eaten. Vitamins and minerals are optional.

The calculation

```
Nutrient contribution = per-100g value x grams eaten / 100
Meal total = sum of reviewed ingredient contributions
Daily total = sum of meals logged for the selected day
```

For example, 200g cooked white rice at 130 kcal/100g contributes 260 kcal. A 150g roasted chicken breast at 165 kcal/100g contributes 247.5 kcal. Together: 507.5 kcal, using the supplied generic reference values. Displayed totals round to one decimal place.

Tracked values	Units / meaning
Energy and macros	Calories in kcal; protein, carbohydrates and fat in grams.
Other nutrients	Fiber and sugar in grams; sodium, calcium, iron, potassium and vitamin C in milligrams.
Selected vitamins	Vitamin A in micrograms RAE; vitamin D and B12 in micrograms.
Missing information	A dash means unknown. A greater-than-or-equal indicator means only known contributions are included; missing values are not assumed to be zero.

Estimates have limits

The 24-food starter list is small and generic. Brands, recipes and cooking methods differ. An arbitrary food name cannot produce reliable nutrition without reference values. Photo guesses cannot measure hidden oil, exact weight or micronutrients. Use labels or recipes for closer estimates.

06 / PHOTO ASSISTANCE

From photo to reviewed meal

You choose which recognition path to use. Nothing is logged automatically.

Path	Requirements	Result
Manual / local	No key, model or internet needed.	You supply foods, grams and label/recipe values.
Online OpenAI	Internet and an OpenAI API key.	Suggested visible foods, portions and per-100g estimates for review. API usage may be billed.
Local classifier	A real-food artifact trained with the ML pipeline.	Suggested dominant-food labels and model scores; you supply grams and nutrition. Not connected in the current package.

Online workflow

1. Choose a JPEG, PNG or WebP photo or use the camera.
2. Choose Connect AI and enter your own key for this session.
3. Choose Analyze photo with AI to send the selected resized photo.
4. Review foods, edit grams and nutrition, remove mistakes, and add missed items such as oil or sauces.
5. Confirm meal name, meal type and journal date; choose Log meal as consumed.

The result is added to the food journal, daily totals and consumed-food activity feed. View items shows logged ingredients. The original photo is not part of the journal or its backup. Camera permission depends on browser/device support.

Connection and lifecycle

A key entered in the browser stays in memory until refresh or locking; it is not persisted or backed up. Developers can use OPENAI_API_KEY in the server environment. The current default server model is gpt-4.1-mini, configurable with BLOOM_AI_MODEL. A ChatGPT login does not configure the API connection.

Current acceptance limit

The provider request/response contract is tested with mocked responses. Actual recognition quality and live provider access still need verification with a real API key and meal photos. Choose manual entry if the connection, credits or recognition result is unsuitable.

07 / PRIVACY & SECURITY

Where information goes

The journal is local. Optional online analysis and explicit training exports have separate data flows.

Data	Location and transfer
Guest journal	Browser storage, unencrypted. Guest backup is ordinary JSON.
Account journal	Encrypted in browser storage. Portable encrypted .bloom backup when downloaded.
Selected photo	Browser memory. Sent to the local server for requested recognition; online Analyze forwards it to OpenAI. Not automatically saved to disk.
AI key	Session memory or developer-configured server environment; used by the local bridge. Not included in journals or backups.
Reviewed training example	Only after explicit Export: an unencrypted JSON download with the resized photo, label, hash and model provenance.
ML experiments	Local run folders by default. A remote MLflow tracking URI explicitly uploads experiment artifacts.

Account protection

Accounts derive an encryption key using Web Crypto PBKDF2-SHA-256 with 600,000 iterations and a random salt. AES-256-GCM uses a fresh random IV per save. The derived key stays in memory and is not persisted. This protects stored account contents; it does not replace device security or an independent security audit.

Server boundaries

The Python server binds to 127.0.0.1, checks Host and Origin for photo requests, requires an application request header, enforces size limits, restricts concurrent analyses, and sends no-store and security headers. Request logs exclude photos, keys and journal content. Do not expose this development server directly to the internet.

The online bridge requests store:false from OpenAI, but provider data handling still applies to photos you choose to send. No cloud request occurs just from choosing a photo. Copy only the app ZIP to a friend unless you deliberately intend to transfer personal data too.

08 / IMPLEMENTATION

How the app is built

A small native web interface, a loopback Python service, and an optional research pipeline.

Part	Responsibility
dist/index.html + design.css	Page shell, access screen, responsive layout, shared visual tokens and dialogs.
app.js + icons.js	Journal views, dates, totals, movement/weight/goals, and local SVG icons.
accounts.js + vault.js	Guest/account lifecycle, storage, encryption, backup validation and migration.
nutrition.js	Starter foods, nutrient schema, portion calculations, missing-data coverage and meal validation.
photo-meal.js + local-model.js	Camera/upload, draft review, online analysis, local prediction UI and deliberate training export.
server.py	Static app serving, OpenAI bridge and optional local classifier endpoints.
ml/	Dataset preparation, baseline/training/evaluation, inference, tests, smoke verification and model documentation.

Local HTTP interface

GET /api/ai/status reports provider configuration. POST /api/ai/analyze accepts a selected JPEG data URL and explicit online consent. GET /api/ml/status reports local model availability. POST /api/ml/predict accepts the selected image and returns a model ID, candidate labels/scores, abstention status, image hash and latency. These are local endpoints, not a hosted public API.

Journal structure

A journal contains a profile name, goals and date-keyed days. Each day contains meals, water, activities and optional weight. Ingredient meals retain per-100g values, grams and source; known nutrient totals are recalculated during import validation. Recognition provenance can include model ID, suggested label, confidence and image hash, without the image.

The browser and server are separate: replacing source files does not copy a user journal. No database service, frontend build, cloud login, analytics service or background training job is required for ordinary use.

09 / ML DATA

Build an honest dataset

Start with a limited classification problem and keep evaluation photos separate from development.

The default ten Food-101 categories are apple pie, chicken curry, fried rice, grilled salmon, hamburger, omelette, pizza, spaghetti bolognese, steak and sushi. These are visual labels, not nutrient profiles. Your own permitted class-folder dataset can be used instead.

Prepare your own images

```
my-food-photos/
  pizza/image-001.jpg
  sushi/image-002.jpg
  ...

python -m ml.data prepare --source C:/datasets/my-food-photos --output ml/data/food-v1.json `
--per-class 100
```

Use at least six independent image groups per class; 50-100+ is a better first experiment. Use --groups with a path,group CSV when multiple pictures show the same meal or source session. The preparer caps the small portfolio subset at 5,000 images.

Leakage controls

- Hash raw bytes and decoded pixels, remove exact duplicates, and group near duplicates with perceptual hashes.
- Keep entire groups in one split; stratify class groups into approximately 70% train, 15% validation and 15% test. Reject conflicting group labels for review.
- Record image identity, class, group, split and seed in a manifest. Verify image contents again before training. Independently inspect source/meal grouping; hashes are imperfect.

Food-101 alternative

The optional food101 command explicitly downloads the full archive, roughly 5 GB, before selecting a small subset. Only official training images are sampled; the internal holdout is not the official benchmark. Dataset images and pretrained weights are not in the share ZIP. Check the upstream image rights before redistribution or commercial use.

Current data status

Network restrictions prevented dataset downloads during implementation. No real-food accuracy result exists yet. Training on synthetic fixtures verified execution only; it did not create a useful food model.

10 / ML TRAINING

Train, compare, evaluate

Measure a baseline first, select settings on validation, and reserve test data for assessment.

Training sequence

The classical baseline uses normalized per-channel color histograms and logistic regression. The vision model uses ResNet18 with ImageNet weights by default. Its first epoch trains the head with a frozen backbone and BatchNorm statistics; subsequent epochs fine-tune the full network with a lower learning rate.

```
python -m ml.train --manifest ml/data/food-v1.json --output ml/runs/food-v1 --epochs 5 `
--freeze-epochs 1 --device cpu
```

The first pretrained run downloads roughly 45 MB of ImageNet weights. The best checkpoint is selected by validation macro F1. A validation-selected temperature adjusts model scores, and an abstention threshold targets at least 80% validation accuracy on accepted predictions with at least 10% coverage. If none qualifies, the model abstains. This is not a future accuracy guarantee.

Metric	Why it matters
Accuracy / macro F1	Overall correctness / equal emphasis on each class. Read alongside per-class support and errors.
Top-3 accuracy	Whether the true label appears among three suggestions. Uninformative with three or fewer classes.
Log loss / calibration error	How predicted scores align with outcomes; still no guarantee on unfamiliar foods.
Coverage / accepted accuracy	How often the model accepts a prediction and how accurate accepted predictions were. Read together.
Confusion matrix / failures	Which classes are confused and what errors need investigation.
Latency	Warmed-up batch-one forward time in reports; application timing also includes preprocessing and first-load overhead.

Seed, configuration, dependency versions, code/data/weight hashes and history are recorded. Deterministic operations are requested, but exact numerical equality across hardware and library versions is not guaranteed. Keep the manifest fixed when comparing one change at a time.

11 / ML INTEGRATION

Serve and improve deliberately

A trained artifact can run locally. Human corrections become training data only through an explicit export.

Experiment outputs

Each run writes REPORT.md, metrics.json, confusion-matrix.csv, failures.json, history.json, config.json and manifest.json. The artifact folder contains metadata.json and weights.pt. Metadata records classes, preprocessing, calibration, version, test metrics and hashes. Tensor-only loading uses weights_only=True.

Connect a real-food artifact

```
python server.py --model-dir ml/runs/food-v1/artifact
```

Stop the previous server before restarting. Use Photo meal > Check model to see availability, classes and held-out metrics. Recognition uses CPU in the local service and does not download weights. The app checks format and weight integrity. Synthetic functional models are refused.

Correct a prediction

Choose a suggestion or enter a different food. Confirm grams and nutrient values before adding the ingredient. A low-score result can abstain; even a high score can be wrong outside the supported classes. The classifier handles one dominant food, not every food on a mixed plate.

Before logging/resetting the draft, Export reviewed training example downloads a *.bloom-review.json with the selected resized JPEG, its hash, your reviewed label and model provenance. It contains no rest-of-journal or account data. It is unencrypted and never triggers automatic retraining.

```
python -m ml.data import-reviews --source C:/datasets/bloom-exports --output ml/data/reviewed

python -m ml.evaluate --model-dir ml/runs/food-v1/artifact --source `
C:/datasets/new-food-photos --output ml/runs/food-v1-external.json
```

Resolve label conflicts before retraining. External evaluation rejects exact/perceptual overlap with the experiment and reports missing classes; inspect meal/source independence yourself. It does not establish out-of-distribution detection. MLflow tracking is optional; JSON/CSV tracking works without it. A remote tracking URI uploads artifacts deliberately.

12 / VERIFIED & PENDING

Evidence, not promises

Verification reflects the local implementation session on 08 November 2026.

Area	Evidence / current limit
Local journal	Vault and nutrition Node checks passed: isolation, encryption round trips, wrong passwords, tampering, storage failures, stale edits, portion math and unknown nutrients.
Python checks	Seven combined ML/server tests passed: grouping, duplicate removal, integrity, metrics, abstention, reviewed export validation and endpoint protections.
Training pipeline	Two-epoch CPU smoke run on 30 synthetic images: 18 train / 6 validation / 6 test. Checkpoint selection, baseline, reports, reload and inference completed.
Browser integration	Separate synthetic fixture exercised local prediction, correction, export/import, 150g Orange totals, consumed activity and persistence. This was not food-recognition evaluation.
Responsive UI	Seven routes checked at desktop/mobile; fixed login/signup tested from 320x568 through desktop and landscape. New model panel checked at 390x844. Muted palette changes only visual tokens.
Online inference	Provider request/response handling mocked. Live provider access and meal recognition accuracy not verified.
Not yet exercised	Real-food training, external food-photo accuracy, GPU runs, MLflow, GitHub CI execution and fresh cross-platform installs.

Known environment limitation

The existing CPU Python installation ran the smoke tests but emitted a torchvision native-image-extension compatibility warning. The pipeline uses Pillow image loading. Use the documented isolated ML environment for a fresh training setup.

A GitHub Actions workflow is included for journal checks, ML unit tests and a synthetic smoke run. Its presence is not proof of a successful hosted CI run. Automated security checks do not replace an independent audit.

13 / CAREER & NEXT STEPS

Turn the app into experience

The strongest portfolio story is a reproducible result with clear scope, ownership and failure analysis.

Skill	Evidence you can show
Data engineering	Dataset manifest, hashes, group-aware splits, class counts and duplicate handling.
Computer vision	Baseline versus transfer learning, augmentation, freeze/fine-tune stages and checkpoint selection.
ML evaluation	Macro F1, per-class scores, calibration, abstention, confusion matrix and failure reports.
ML engineering	Versioned artifacts, validated local inference API, latency and reproducible commands.
Product development	A working interface for review, corrections, nutrient logging, account separation and backups.
MLOps / responsibility	Experiment records, optional MLflow, CI configuration, explicit data exports and documented limits.

Next work in priority order

1. Collect permitted real-food photos with known source/meal groups.
2. Run the ImageNet-initialized baseline comparison and inspect errors.
3. Evaluate genuinely new photos and unfamiliar inputs.
4. Complete the model card with measured metrics and limitations.
5. Connect the selected artifact and test the actual product flow.
6. Only then consider multi-food detection, segmentation or portion-estimation research.

Describe the current project truthfully

You can say that you built an integrated local nutrition journal and implemented a reproducible food-classification pipeline with evaluation and human review. You cannot yet claim measured real-food accuracy, a medically validated calorie estimator or a production-deployed model.

For an eventual resume, replace placeholders with actual results: "Built an end-to-end food classification system using PyTorch and ResNet18, compared it with a classical baseline, achieved [measured macro F1] on [described held-out set], and integrated reviewed predictions into a local nutrition journal." Explain your own contributions and AI-assisted work clearly.

14 / TROUBLESHOOTING

When something looks wrong

Start with the local address, browser profile and selected journal date.

Problem	What to check
App will not open	Extract the ZIP first. Run Open Bloom.cmd with Python installed. Keep the server window open and visit http://localhost:4317.
Port already in use	Close the previous Bloom server, then launch again. A different --port is possible, but a different address/port uses separate browser storage.
Journal seems missing	Use the same browser profile, address and port, then choose the same guest/account and date. localhost and 127.0.0.1 are different storage origins.
Forgot the password	There is no reset service. An encrypted backup still requires its original password. A new account cannot unlock old ciphertext.
Photo analysis unavailable	For online AI, connect your own valid API key and check internet/credits. For local recognition, train and connect a real artifact. Manual entry remains available.
Camera not available	Allow camera access in the browser, check device support, or use Choose photo instead.
Incorrect food or totals	Review preparation, grams and per-100g values. Add hidden oil/sauces yourself. Remove wrong entries and enter corrected ones.
Friend cannot use your URL	Send the ZIP. Your localhost link points to your own device and is not a public website.
Backup / save error	Preserve the current journal, export if possible, and resolve storage or stale-tab warnings before overwriting anything. Do not clear browser data as a first step.

What Bloom is not yet

It is not an installed native mobile app, cloud-synced account system, wearable integration, clinical nutrition service or released pretrained food model. It records and estimates from reviewed data; it does not determine overall health or prescribe treatment.

15 / REFERENCE

Commands and source map

Run developer commands from the extracted Bloom folder. Ordinary users can skip the ML environment.

Ordinary use and regression checks

```
python server.py
node check-vault.cjs
node check-nutrition.cjs
python -m unittest test_server.py
```

Isolated ML setup and verification

```
py -3.11 -m venv ml/.venv
ml/.venv/Scripts/python -m pip install -r ml/requirements.txt
ml/.venv/Scripts/python -m unittest ml.test_pipeline
ml/.venv/Scripts/python -m ml.smoke --output .qa/ml-smoke-new
```

Use an installed Python 3.11+ version for the optional environment. Torch, torchvision, Pillow, NumPy and scikit-learn are ML dependencies; they are not required just to keep a manual journal. Each smoke/output run needs a fresh directory. See ML-PROJECT.md for the full training, Food-101, external-evaluation and optional MLflow commands.

Project documents and evidence

README.md: quick start. USER-GUIDE.md: accounts, backups and photo/manual entry. ML-PROJECT.md: training and portfolio workflow. ml/MODEL-CARD.md: real-model reporting template. ml/VERIFICATION.md: evidence and unverified acceptance. ASSET-NOTES.md: generated fitness image and transparent cutout. .21st/REDESIGN.md: retrieved design references. Source files and tests are included in the sharing package.

Primary references

PyTorch transfer learning tutorial
Food-101 dataset and provenance
MLflow experiment tracking
21st Health Stat Card - ruixen.ui
21st Sidebar - arunjdass
21st split sign-in - diarmuradi

This guide describes the source snapshot and observed checks, not a guarantee of future service availability. Generated fitness imagery depicts fictional adults. The guide and app ZIP exclude personal journals, passwords, keys, dataset images and trained weights.